Scheduling process of quantum circuits to optimize tasks execution on quantum computers

Javier Romero-Alvarez
Quercus Software Engineering Group
Universidad de Extremadura
Cáceres, Spain
jromero@unex.es
0000-0002-3162-1446

Jaime Alvarado-Valiente
Quercus Software Engineering Group
Universidad de Extremadura
Cáceres, Spain
jaimeav@unex.es
0000-0003-0140-7788

Jorge Casco-Seco
Quercus Software Engineering Group
Universidad de Extremadura
Cáceres, Spain
jcascose@unex.es
0009-0003-9166-6827

Enrique Moguel
Quercus Software Engineering Group
Universidad de Extremadura
Cáceres, Spain
enrique@unex.es
0000-0002-4096-1282

Jose Garcia-Alonso
Quercus Software Engineering Group
Universidad de Extremadura
Cáceres, Spain
jgaralo@unex.es
0000-0002-6819-0299

Juan M. Murillo
Quercus Software Engineering Group
Universidad de Extremadura
Cáceres, Spain
juanmamu@unex.es
0000-0003-4961-4030

2024 IEEE International Conference on Quantum Computing and Engineering (QCE) | 979-8-3315-4137-8/24/$31.00 ©2024 IEEE | DOI: 10.1109/QCE60285.2024.10275

***Abstract*—Quantum computing holds significant potential across various fields such as cybersecurity, finance, and healthcare, among many others. However, we currently face multiple limitations such as the low availability of computers, the underutilization of available qubits, and the ever-increasing number of users, which results in prolonged waiting times. This, combined with the cost of executions, limits access to quantum computing. To address these challenges, we propose a scheduling process to optimize task execution on quantum computers. The proposed scheduling process unifies multiple circuits to enhance the efficiency of resource utilization in quantum machines. This approach allows for better use of available qubits and reduces the associated execution costs. We validate the proposed method employing various quantum algorithms by systematically examining the scheduling process through a comparative case study between two quantum service providers: IBM Quantum and Amazon Braket. This validation has shown that the scheduling process does not generate a significant impact on the results, ensuring similar results to single circuit executions with a fraction of the cost.**

***Index Terms*—Quantum Computing, Quantum Software Engineering, Quantum Circuit Scheduling.**

I. Introduction

Companies such as Amazon, IBM, Microsoft, and Google are investing in the development of increasingly powerful quantum computers. Specifically, IBM has set a date on its roadmap for the 1000-qubit computer, which will mark a milestone in quantum computing [1]. In addition to hardware development, they are also investing in offering quantum computing as a service through their cloud platforms, as if it were just another classic service [2].

However, when accessing these computing services, developers face various limitations. On the one hand, there is a low availability of machines to run their executions, which is due to the small number of existing machines and their variable availability [3]. This results in long queue wait times. On the other hand, and closely related to the first issue, there is the problem of the high execution costs that developers have to pay when running their algorithms on quantum computers [4].

Analyzing the problem of long queue times, we find the underutilization of qubits in quantum machines. Specifically, the average number of qubits used by a circuit is 10.5 [5] even when using machines in the range of one hundred qubits. As a result, a large number of qubits are underutilised in each of the executions launched by the developers.

To optimize the usage of these valuable resources and address the issue of long waiting times, this work proposes a process for quantum circuit scheduling. This proposal aims to reduce waiting times and improve quantum computer utilization by combining multiple developers' circuits into a single execution task, thus making more efficient use of available qubits.

To present this, the rest of the paper is organized as follows: Section II analyzes the background of the present work and discusses the most relevant related works. Section III presents the proposal for the scheduling process. Section IV evaluates the proposal by systematically validating the scheduling process through a comparison between two quantum service provider and the impact of noise on the obtained results. Finally, Section V presents the conclusions and future works.

II. Motivation & Related works

In classical computing, performance challenges have been addressed through parallelization and resource optimization techniques in High-Performance Computing (HPC) [6]. Techniques such as task scheduling, load balancing, and resource allocation have been successfully implemented to maximize the efficiency of computational resources.

In terms of quantum computing, several works are emerging to address these issues. For example, one research proposes

979-8-3315-4137-8/24/$31.00 ©2024 IEEE
DOI 10.1109/QCE60285.2024.10275

Authorized licensed use limited to: CBS Library. Downloaded on March 31,2026 at 20:31:31 UTC from IEEE Xplore. Restrictions apply.

an empirical evaluation to help optimize the resources of quantum computers employing the quantum circuit mutation [7]. Another approaches involve the parallel execution and multi-programming of quantum circuits to reduce idle times and improve throughput [8], [9]. These works differ from our proposal by partitioning the physical qubits to execute several quantum circuits simultaneously on the same set of qubits.

Other significant research efforts focus on optimizing the quantum circuits themselves rather than the quantum tasks [10]. Techniques such as circuit transpilation and gate reduction aim to simplify quantum circuits, reducing their depth and the number of required gates [3].

The proposed scheduler builds on these concepts by adopting a process that enables the scheduling of diverse quantum circuits from multiple developers into a unified execution task.

III. Quantum Circuit Scheduler Process

In this section, the proposed process for optimizing task execution on quantum computers is presented, as illustrated in Figure 1. The process consists of 4 steps:

1) **Scheduling:** The process starts with one or several developers submitting their circuits into the Scheduler API to be executed. Developers must specify in their request the circuit, and the machine on which it is to be executed—either the machines provided by IBM Quantum or Amazon Braket are supported at the moment. The format in which they can submit circuits is via the URL generated by the Quirk circuit simulator[1], or via the URL of the repository where the Python code for the circuit is hosted, in the Qiskit or Braket languages [11].
   The received circuits are stored in the execution queue of the Scheduler to perform the executions in cycles. A cycle is defined as a finite period of time in which the Scheduler waits to receive more circuits for scheduling or until the number of available qubits is completed. Each time it completes one, the system goes through the elements in the queue and, as long as there are qubits available on the machine where it will be executed, schedules them in a single circuit. The selection of the circuits in the queue follows the *First In, First Out* FIFO method.
   In this process, the code of the circuit is modified to specify on which qubits of the machine it will be executed. This means that if, e.g., we have to plan two circuits that use 3 and 4 qubits respectively, the scheduler modifies the code to indicate that the first one will use qubits from 0 to 2, and the second one from 3 to 6.
2) **Execution:** Once a cycle has been completed, and several circuits have been scheduled, they are launched for execution. To do this, the scheduled circuit for the selected machine is transpiled and sent to the execution queue of the machine provided by the cloud provider.
   During transpilation, the quantum compiler reorganizes the circuit gates to adapt them to the computer topology, which may involve the introduction of additional gates to establish connections between qubits. However, although transpilation reorganizes the gates of the circuit to adapt them to the topology, it does not affect the qubits on which the results are obtained, as they have been previously assigned for each circuit, but it can affect intermediate points in the circuit.
3) **Unscheduling:** When the execution is finished, it goes to the unscheduling part of the obtained results. The

[1]https://algassert.com/quirk

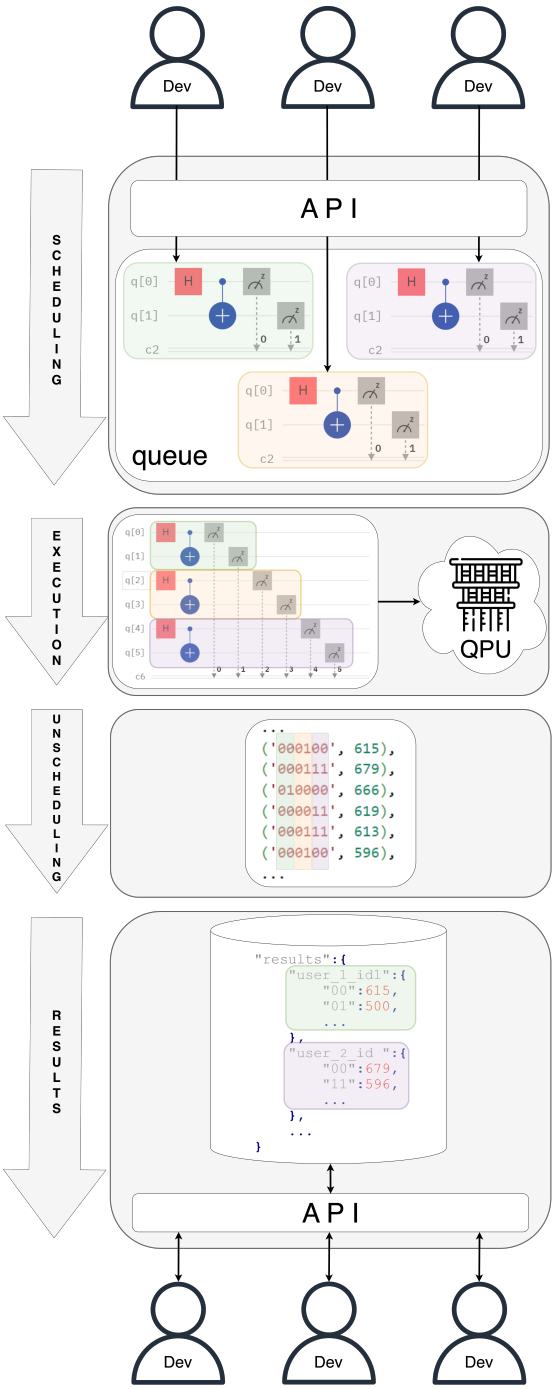

Figure 1: Quantum circuit scheduler process

Authorized licensed use limited to: CBS Library. Downloaded on March 31,2026 at 20:31:31 UTC from IEEE Xplore. Restrictions apply.

obtained results from the executions are a combination of all the results of each of the scheduled circuits. Therefore, it is necessary to perform a result unscheduling process to separate the results of each circuit individually. Specifically, it receives as input the set of obtained results from the execution, splits them up, assigns each of the results a unique identifier, and stores each of them in a database.

Following the previous example, where the first scheduled circuit uses the first 3 qubits, and the second uses the next 4 qubits, in the unscheduling process the results are divided by assigning to the first one the values of the first 3 bits, and to the second one the values of the remaining 4 bits.

4) **Return results:** With the unique execution identifier returned to the developer when sending the circuit to the scheduler, developers can request the results. Developers make requests to the scheduler API, where they indicate this identifier and, if the execution has finished, they receive the results in JSON format.

This approach ensures an efficient process for executing quantum tasks, addressing some of the complexities inherent in quantum computing. Such as computer underutilization and high waiting times. However, the proposal needs to be evaluated to verify these aspects.

IV. Evaluation

There are inherent challenges associated with quantum hardware, such as qubit decoherence, gate errors, and environmental noise, which impact the reliability of quantum computations [12]. Noise in quantum computing refers to any unwanted interactions that disturb the state of qubits, leading to errors in the results of quantum computations. For these reasons, it is necessary to perform a validation of the proposal to verify that the obtained results in the execution after the scheduling process are not significantly affected by noise, although it is expected that quantum computers will generate progressively less noise, or that noise will be reduced by the application of e.g. AI-based models such as those of IBM [13].

In this section, we evaluate the proposal by systematically validating the scheduling process through a comparison between two quantum service providers: IBM Quantum and Amazon Braket. For this analysis we use a case study employing several quantum algorithms widely know in the scientific community, which demonstrates the effectiveness and robustness of the presented approach. In addition, we evaluate the impact of noise on the obtained results in the different executions in both providers.

A. Case study comparison between quantum service providers

In our case study, the quantum service providers IBM Quantum and Amazon Braket were chosen. These providers were chosen because they offer the most up-to-date quantum computers. IBM Quantum, in particular, stands out for providing machines with a large number of qubits, currently reaching 156 qubits.

In addition to the use of different providers, the use of several processors with different topologies and numbers of qubits strengthens the validation of our proposal, as it demonstrates the adaptability of the process in different hardware configurations.

For Amazon Braket, it was selected the Quantum Processing Unit (QPU) “IQM Garnet” of 20 qubits. In the case of IBM Quantum, the “Eagle”, “Heron r1”, and “Heron r2” QPUs have been used, containing a total of 127, 133, and 156 operational and connected qubits, respectively. Because some of these processors only recognize limited types of gates, the sheduled circuits are matched to the available connectivity and gates. In this way, each scheduled circuit is assigned to a specific set of qubits. Ensuring that the qubits used by a circuit are not shared with other circuits.

It should also be noted that the selected providers do not present a scheduling of the circuits in the way shown in our proposal. They offer task scheduling with user-defined calendars or allow scheduling of a reservation so that the user knows exactly when the execution of his workload starts and ends.

In our study, we utilized a collection of quantum circuits available in the Zenodo public repository[2]. This repository includes various implementations of circuits associated with significant algorithms in the quantum computing field, along with their characteristics. For the validation tests, we selected a total of 10 algorithms and their respective implementations. The specific algorithms used were: Berstein-Vazirani, Deutsch-Jozsa, Full Adder, Grover, Kickback, Phase Estimation, Quantum Walk, Shor, Simon, and Travelling Salesman Problem (TSP). These algorithms have been sent to the scheduler to be executed with different shot configurations, resulting in the scheduling of different tasks.

After executing the scheduled tasks on the processors of both providers, a considerable reduction in execution costs was obtained. Specifically, in the case of Amazon Braket, an average reduction of 75% in execution costs was achieved, reducing from $44.40 to $11.10. In the case of IBM Quantum, the average reduction was even higher, 93.53%, reducing from $363.08 to $23.27.

The execution cost of each circuit was calculated by obtaining the proportional part of the total execution cost, taking into account the number of qubits used in the total scheduling. The average sum of the time of the individual executions in both providers was 80 seconds, while the scheduling took 20 seconds. The circuit execution time in scheduling is very close to the time it takes to execute the deepest circuit. Thus, it has been possible to greatly reduce this time, which allows a significant cost reduction.

These results allow us to conclude that circuit scheduling significantly improves queuing times and reduces execution costs. Furthermore, it was possible to execute the different scheduled tasks using capacities close to 100% of the QPUs. However, despite these aspects, it is still necessary to verify that the obtained results after the execution of the scheduled

[2]https://doi.org/10.5281/zenodo.11633677

Authorized licensed use limited to: CBS Library. Downloaded on March 31,2026 at 20:31:31 UTC from IEEE Xplore. Restrictions apply.

tasks are not affected by noise.

B. Impact of noise on execution results

To ensure that the obtained results from the scheduled circuits are not significantly affected by the inherent noise and decoherence of quantum computers in the current NISQ era, it is essential to evaluate this impact. In this context, a statistical study was performed to compare the obtained results from each individually executed circuit with those obtained by scheduling.

For this comparison, we used a statistical algorithm that measures the distance between probability distributions, the *Wasserstein Distance* algorithm [14]. This algorithm quantifies the effort required to transform one distribution into another, providing a value between 0 and 1 that indicates the similarity between two distributions; the smaller the value, the greater the similarity.

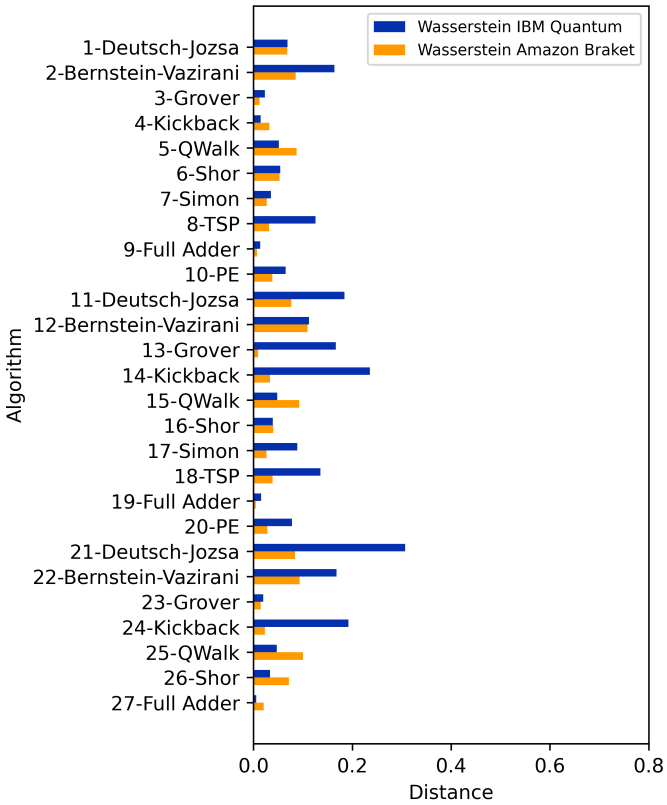

Figure 2: Statistical analysis of probability distribution

Figure 2 presents the distance between the different scheduled and individual executions using the *Wasserstein Distance* algorithm. The orange bars represent the distance according to this algorithm for the Amazon Braket executions and the blue bars for IBM Quantum. There are some algorithms that are repeated because they have been used in different scheduling test compositions. So, it can be seen that there is a difference in the results compared to the individual executions due to noise. This difference does not exceed 0.4 in any case, and in general the percentage difference is only 7%, with cases such as Grover and Full Adder where the difference is practically zero. This suggests that there is an impact on noise, although it is small compared to the savings in running costs.

In other cases, such as with the Deutsch-Jozsa and Berstein-Vazirani algorithms, the difference is slightly larger, so it is inferred that there is more noise. This is due to the circuit design of these algorithms, which consist of a large number of CNOT gates. These gates are essential for creating entanglement and performing logic operations in quantum algorithms but are the most susceptible to being affected by noise [15].

On the other hand, the results on Amazon Braket present in general lower noise, because the executions have been performed on a QPU with a significantly lower number of qubits concerning those of IBM Quantum. Smaller quantum processors have a limited number of qubits compared to larger quantum processors. With fewer qubits, there are fewer interactions between them, which can reduce the occurrence of errors and noise due to decoherence and other sources of interference.

All these results allow us to conclude that circuit scheduling not only significantly improves queuing times and optimizes machine capacity utilization, but also demonstrates that noise does not seriously affect the obtained results. As can be seen from the low distance observed between the scheduled and individual executions in the statistical analysis performed.

Even if there are cases where the scheduling results have suffered a higher amount of noise, either because of the circuit design or the number of qubits in the machine, it is still possible to identify which is the correct value. Therefore, the scheduling proposal will be valid and scalable for future machines with more qubits and different configurations, ensuring applicability as quantum technology advances.

The results of all executions are available in the noise-validation folder of the Zenodo repository[3].

V. Conclusion & Future Work

The work presented focuses on optimizing the use of quantum resources through a scheduling process that schedules multiple user circuits on a single execution task. This approach aims to reduce the waiting times and operational costs associated with the use of quantum computers. Specifically, the overall average execution time on the providers used in the validation was reduced from 80 seconds for individual runs to 20 seconds for scheduling. Also, an average reduction in execution costs of 75% on Amazon Braket and 93.53% on IBM Quantum was achieved.

The main challenge of the study has been to manage the errors that can occur in the results due to noise, but as it has been found, the distribution of results in scheduling versus individual execution is very similar.

Specifically, validation of the scheduling process was performed by running known quantum algorithms on scheduled tasks compared to single executions. In addition to performing different executions before and after the transpilation processes. The obtained results show that noise and decoherence

[3]https://zenodo.org/doi/10.5281/zenodo.11633677

Authorized licensed use limited to: CBS Library. Downloaded on March 31,2026 at 20:31:31 UTC from IEEE Xplore. Restrictions apply.

do not increase significantly when running combined circuits, suggesting that the proposed scheduling process is effective and does not compromise the integrity of the results.

Currently, once the proposal has been validated, we are working on extending the proposal to more providers, and to the design of different policies for circuit scheduling, where developers will be able to choose for example to have better execution times, execute with circuits of the same depth to reduce noise, or even select the exact number of shots they want to execute.

ACKNOWLEDGMENTS

This work has been partially funded by the European Union "Next GenerationEU /PRTR", by the Ministry of Science, Innovation and Universities (TED2021-130913B-I00, RED2022-134148-T, and PDC2022-133465-I00). It is also supported by QSERV project funded by the Spanish Ministry of Science and Innovation and ERDF; by the Regional Ministry of Economy, Science and Digital Agenda of the Regional Government of Extremadura (GR21133); and by European Union under the Agreement - 101083667 of the Project "TECH4E -Tech4effiency EDIH" regarding the Call: DIGITAL-2021-EDIH-01 supported by the European Commission through the Digital Europe Program. It is also supported by grants PRE2022-102070 and PID2021-1240454OB-C31, financed by MCIN/AEI/10.13039/501100011033, "FEDER/EU", and by FSE+.

REFERENCES

[1] C. G. Almudever, R. Wille, F. Sebastiano, N. Haider, and E. Alarcon, "From designing quantum processors to large-scale quantum computing systems," in *2024 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 2024, pp. 1–10.

[2] J. Romero-Álvarez, J. Alvarado-Valiente, E. Moguel, J. Garcia-Alonso, and J. M. Murillo, "Enabling continuous deployment techniques for quantum services," *Software: Practice and Experience*, 2023.

[3] P. Das, S. S. Tannu, P. J. Nair, and M. Qureshi, "A case for multi-programming quantum computers," in *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*, ser. MICRO '52. New York, NY, USA: Association for Computing Machinery, 2019, p. 291–303.

[4] J. Alvarado-Valiente, J. Romero-Álvarez, E. Moguel, J. Garcia-Alonso, and J. M. Murillo, "Technological diversity of quantum computing providers: a comparative study and a proposal for api gateway integration," *Software Quality Journal*, pp. 1–21, 2023.

[5] T. Ichikawa, H. Hakoshima, K. Inui, K. Ito, R. Matsuda, K. Mitarai *et al.*, "A comprehensive survey on quantum computer usage: How many qubits are employed for what purposes?" *arXiv:2307.16130*, 2023.

[6] R. P. Garg, I. A. Sharapov, and I. Sharapov, *Techniques for optimizing applications: high performance computing*. Sun Microsystems Press Palo Alto, 2002.

[7] E. M. Usandizaga, T. Yue, P. Arcaini, and S. Ali, "Which quantum circuit mutants shall be used? an empirical evaluation of quantum circuit mutations," *arXiv preprint arXiv:2311.16913*, 2023.

[8] L. Liu and X. Dou, "Qucloud+: A holistic qubit mapping scheme for single/multi-programming on 2d/3d nisq quantum computers," *ACM Transactions on Architecture and Code Optimization*, vol. 21, no. 1, pp. 1–27, 2024.

[9] S. Niu and A. Todri-Sanial, "Enabling multi-programming mechanism for quantum computing in the nisq era," *Quantum*, vol. 7, p. 925, 2023.

[10] B. Nash, V. Gheorghiu, and M. Mosca, "Quantum circuit optimizations for nisq architectures," *Quantum Science and Technology*, vol. 5, no. 2, p. 025010, 2020.

[11] J. Romero-Álvarez, J. Alvarado-Valiente, E. Moguel, J. García-Alonso, and J. M. Murillo, "Using open api for the development of hybrid classical-quantum services," in *International Conference on Service-Oriented Computing*. Springer, 2022, pp. 364–368.

[12] F. Leymann and J. Barzen, "The bitter truth about gate-based quantum algorithms in the nisq era," *Quantum Science and Technology*, vol. 5, no. 4, p. 044007, 2020.

[13] A. Shaib, M. H. Naim, M. E. Fouda, R. Kanj, and F. Kurdahi, "Efficient noise mitigation technique for quantum computing," *Scientific Reports*, vol. 13, no. 1, p. 3912, 2023.

[14] Y. Cai and L.-H. Lim, "Distances between probability distributions of different dimensions," *IEEE Transactions on Information Theory*, vol. 68, no. 6, pp. 4020–4031, 2022.

[15] M. Urbanek, B. Nachman, V. R. Pascuzzi, A. He, C. W. Bauer, and W. A. de Jong, "Mitigating depolarizing noise on quantum computers with noise-estimation circuits," *Physical review letters*, vol. 127, no. 27, p. 270502, 2021.

Authorized licensed use limited to: CBS Library. Downloaded on March 31,2026 at 20:31:31 UTC from IEEE Xplore. Restrictions apply.